

ECE/CS 552 Introduction to Computer Architecture
University of Wisconsin-Madison
Fall 2024
Boone Severson

Midterm Exam

Thursday, October 24, 2024
75 Minutes (9:30-10:45AM), In-Class

1. The exam is two-sided and has 14 numbered pages, which includes one blank page, one page of blank pipeline diagrams and the two-page MIPS Green Card in the end.
2. Closed book/closed notes examination.
3. No handheld computers or portable computers allowed.
4. Calculator allowed.
5. A single 8.5"x11" double-sided, hand-written cheat sheet is allowed.
6. When applicable, show your work. Any partial points will be awarded based only on what appears in your answers.
7. No one should leave the room during the last 5 minutes of the examination.
8. Upon announcement of the end of the exam, stop writing on the exam paper immediately. Wait for proctors to pick up all exams. The instructor will announce when you may leave the room.
9. Failure to follow instructions may result in forfeiture of your exam and will be handled according to UWS 14 academic misconduct procedures.

FIRST (GIVEN) NAME: _____

LAST (FAMILY) NAME: _____ **SOLUTION** _____

STUDENT ID#: _____

	Achieved Marks	Maximum Marks
Problem 1		/ 12
Problem 2		/ 6
Problem 3		/ 8
Problem 4		/ 6
Problem 5		/ 10
Problem 6		/ 6
Problem 7		/ 8
Problem 8		/ 8
Total		/ 64

Problem 1 (12 marks)

Consider the following MIPS ISA assembly from our class material. **Assume variables f, g, h, i are stored in \$s0, \$s1, \$s2, and \$s3 respectively**, and their initial values are not known. Convert the following C expressions into corresponding **core instruction set** MIPS instructions (left column of MIPS Green Card Page 1), **no pseudo-instructions allowed** for this problem. These expressions are all independent; there is no continuity between the sub-sections. Try to minimize the lines of assembly used.

(a) (2 marks) $f = g*4 + i$

```
sll $s1, $s1, 2      # g<<2 is the same as g*4
add $s0, $s1, $s3
```

or

```
add $s1, $s1, $s1     # g=g+g (aka 2*g)
add $s1, $s1, $s1     # g=2g+2g
add $s0, $s1, $s3
```

(b) (4 marks) **if (h >= i) { h += i; }**

```
slt $t0, $s2, $s3     # h<i is !(h>=i)
bne $t0, $zero, SKIP  # skip
add $s2, $s2, $s3
SKIP:
```

(c) (6 marks) **for (i=0, i<10; i++) {
 f += i;
}**

```
add $s3, $zero, $zero  # i = 0
addi $t0, $zero, 10    # load 10 into $t0
LOOP: beq $t0, $s3, DONE
add $s0, $s0, $s3
addi $s3, $s3, 1
j LOOP
DONE:
```

Problem 2 (6 marks)

Consider the following instruction in our **MIPS ISA** from class:

L1: `sll $s2, $t6, 13`

(a) (2 mark) Circle which of the Basic Instruction Formats this instruction maps to:

R-Format

I-Format

J-Format

`sll` is an R-format instruction, $R[rd] = R[rt] \ll \text{shamt}$;

(b) (4 marks) Provide the **32-bit machine encoding** of this instruction **in binary**. Feel free to use a gap or _ (underscore) in the bits to separate fields for visual clarity. Any unused fields should be set to 0.

`000000_00000_01110_10010_01101_000000`

opcode rs rt rd shamt funct

opcode/funct 0/0

rs unused

rt \$t6 or \$14

rd \$s2 or \$18

shamt 13

Problem 3 (8 marks)

You're working to improve performance on this workload, and the team has settled on two ideas that offer a better balance over the current architecture. The first proposal has a 6x improvement in matrix multiplication and 3x in memory read. The other idea has 5x improvement in matrix multiplication and 5x in memory read. Which should you choose?

Operations	Fractional Time
Matrix Multiplication	58%
Other Arithmetic	8%
Memory Read	32%
Memory Write	2%

Proposal 1:

m=multiplication, r=reads

$$\frac{1}{1 - m - r + \frac{m}{6} + \frac{r}{3}} = \frac{1}{1 - .58 - .32 + \frac{.58}{6} + \frac{.32}{3}} 3.297x$$

Proposal 2:

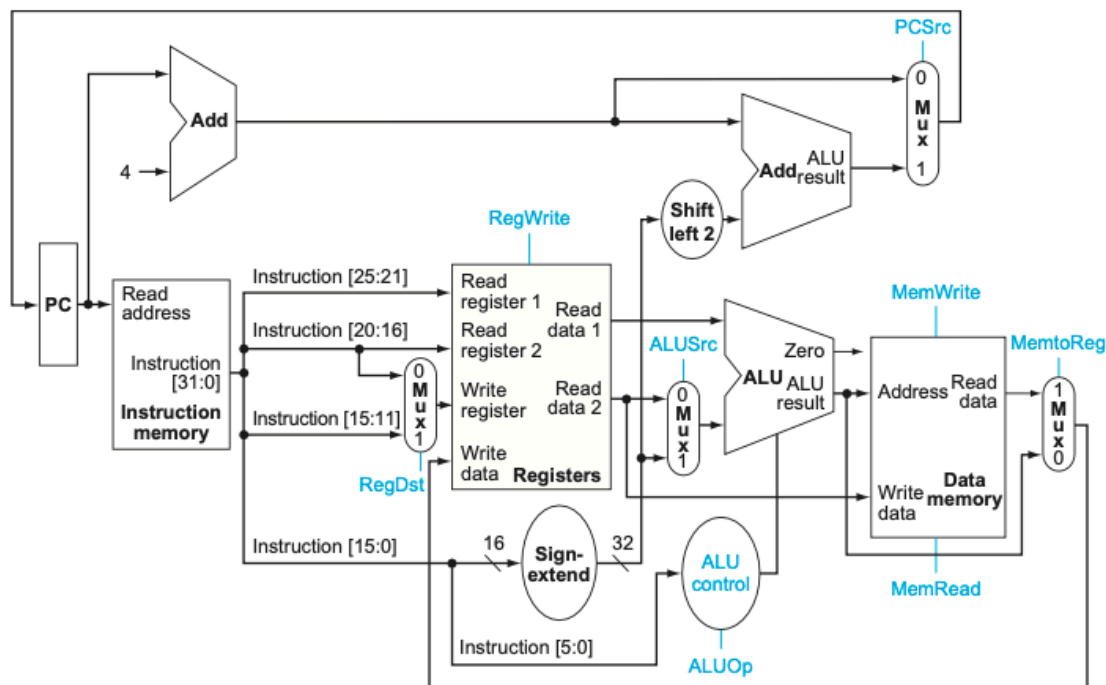
$$\frac{1}{1 - m - r + \frac{m}{5} + \frac{r}{5}} = 3.571x$$

Proposal 2 is better.

Problem 4 (6 marks)

Your processor executes an important workload that frequently packs together the upper 16 bits of each of 2 registers, (pack rd, rs, rt), to save on data memory storage. In this single-cycle processor (CoD figure 4.15), you add an R-format instruction to decrease the number of instructions needed to perform the work. The operation, using MIPS Green Card Verilog format, looks like this:

$$R[rd] = \{R[rs][31:16], R[rt][31:16]\}$$



No changes to the processor data path are required for this instruction; changes are hidden inside ALU control and the ALU itself. Fill in the values of the control signals in the tables below

Control Signal	Value (0, 1 or X)
RegWrite	1
RegDst	1
ALUSrc	0

Control Signal	Value (0, 1 or X)
MemWrite	0
MemRead	0 (or X)
MemtoReg	0

Regular R-Format instruction that reads rs/rt, sends both to ALU, writes ALU result to rd.

Problem 5 (10 marks)

Consider the following MIPS assembly program that contain register data dependences (RAW, WAW, WAR):

```
L1: add $t0, $s0, $t3
L2: lw $t2, 2($t0)
L3: nor $t3, $t2, $zero
L4: addi $t2, $t3, 1
L5: sw $t2, 0($t1)
```

(a) (2 marks) Which dependency/dependencies are known as a **true dependency**? Circle all that match:

Read after Write

Write After Write

Write After Read

Read after Write aka RAW

(b) (8 marks) List all of the register data dependences (all 3 types). Starting with the example given, list the remaining.

\$t0, RAW, L1→L2

\$t2, RAW, L2→L3

\$t2, RAW, L4→L5

\$t3, RAW, L3→L4

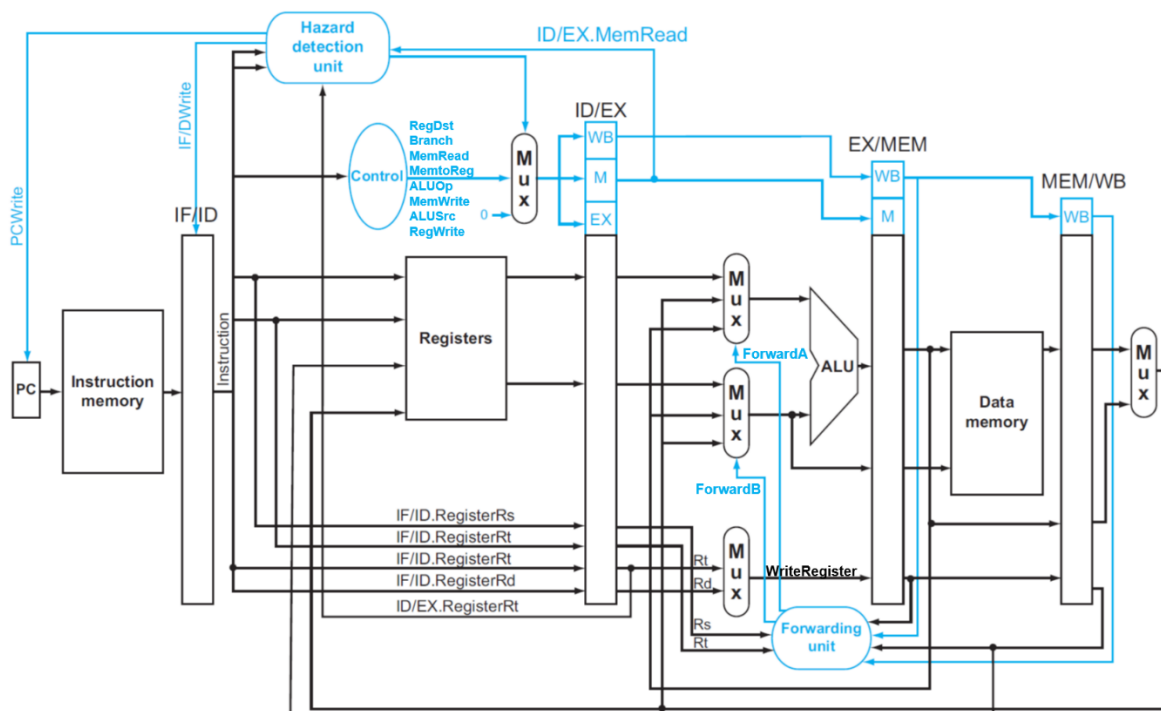
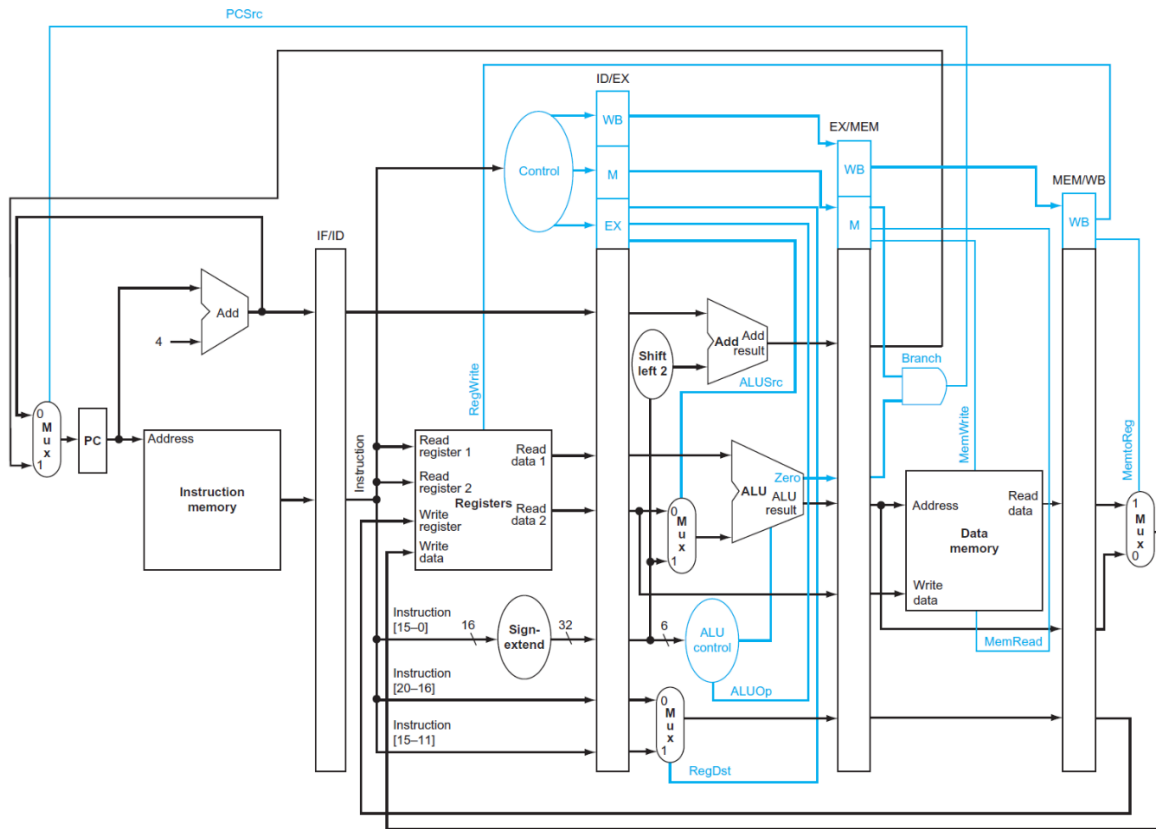
\$t3, WAR, L1→L3

\$t2, WAR, L3→L4

\$t2, WAW, L2→L4 (optional)

Problem 6 (6 marks)

Consider the processor with top diagram (COD Figure 4.51) detailing control signals and bottom (COD Figure 4.60) detailing **Forwarding to EX and RF Bypass**. They are the same CPU.



You run the following MIPS program on this processor.

```
L1: add $s0, $s0, $s1
L2: lw $t0, 12($s0)
L3: sll $t0, $t0, 2
L4: addi $t4, $t0, 1
L5: sw $t0, 4($s0)
```

The following **pipeline diagram** is generated from the execution of the program.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
L1	F	D	X	M	W														
L2		F	D	X	M	W													
L3			F	D*	D	X	M	W											
L4				F*	F	D	X	M	W										
L5						F	D	X	M	W									

(b) (2 marks) In 1 or 2 sentences, explain why L3 stalls in cycle 4.

L3 has a load-to-use stall on \$t0 in cycle 4 that sets up W→X (L2→L3) in cycle 6.

(c) (2 marks) In 1 or 2 sentences, explain why **EX/MEM.MemtoReg** would be **1'b1**. Then list which cycles, if any, **EX/MEM.MemtoReg == 1'b1**?

EX/MEM.MemtoReg==1 is when an **lw** is in MEM stage, as MemtoReg was computed in decode but must “ride along” with the instruction in pipeline registers until it is used in the WB stage. L2 lw is in M in cycle 5.

(d) (2 marks) In one or 2 sentences, explain why **EX-to-EX forwarding** would be enabled. Then list which cycles, if any, **EX-to-EX** forwarding is **enabled** in this program?

EX-to-EX forwarding is enabled when a result from an ALU is in M and the instruction in X is using it as a source/input. This happens in cycle 4 (L1→L2) and cycle 7 (L3→L4).

Problem 7 (8 marks)

Consider a direct-mapped data cache for the 32-bit MIPS processor in our course material. It contains **8 KiB of data storage** with **8-word cache blocks** (aka *cache lines*) and has a **write-through policy**.

(a) (4 marks) How many **total bytes** are required to be stored in the cache, including both the data itself and any metadata?

The MIPS 32-bit processor has 32-bit addresses and 32-bits (**4 bytes**) **per word**, so 8 words equals **32 bytes per cache block**. The byte offset is $\log_2(32 \text{ bytes}) = 5$ bits. The number of indices is 8 KiB/32B or **256**, and $\log_2(256)$ is 8 bits. With 8+5 address bits implicitly stored, this leaves 19 bits for the tag. Finally, a write-through cache only needs a valid bit and not a dirty bit.

Total cache storage: 8 KiB Data + 19*256 bits for **tag** + 1*256 bits for **valid** (8 bits per byte)
 $= 8,192 + 4864/8 + 256/8 = 8,832 \text{ bytes}$

(b) (4 marks) Consider the following benchmark implemented in C code executing on a system with the data cache described above. When this code starts executing, the data cache is 100% empty. &array[0] is perfectly aligned with the beginning of a cache block. Count the number of misses, hits due to temporal locality, and hits due to spatial locality.

```
int array[100];                                misses: 2
for(int i=0; i<7; i++) {
    ... = array[2*i];                          temporal locality hits: 6
    ... = array[2*i+2];                        spatial locality hits: 6
}
```

Cache has 8 words per block. All misses will be cold/compulsory misses as the cache is initially empty. In every iteration, the words at $2*i$ and $2*i+2$ is loaded. When i is 0, a miss happens at $[2*0]$ but has a spatial hit on $[2*0+2]$ because the miss brought the full cache block in (words 0-7). On the next loop, i is 1, so $[2*1]$ has a temporal hit (because $i==0$ already read this as $[2*0+2]$) and $[2*1+2]$ has a spatial hit. $i==2$ is the same, but $i==3$ is slightly different as $[2*3]$ has a temporal hit but $[2*3+2]$ has a miss because it's in the next cache block. An illustration combining iteration (e.g. $i==0$) and result (Miss, Temporal hit, Spatial hit):

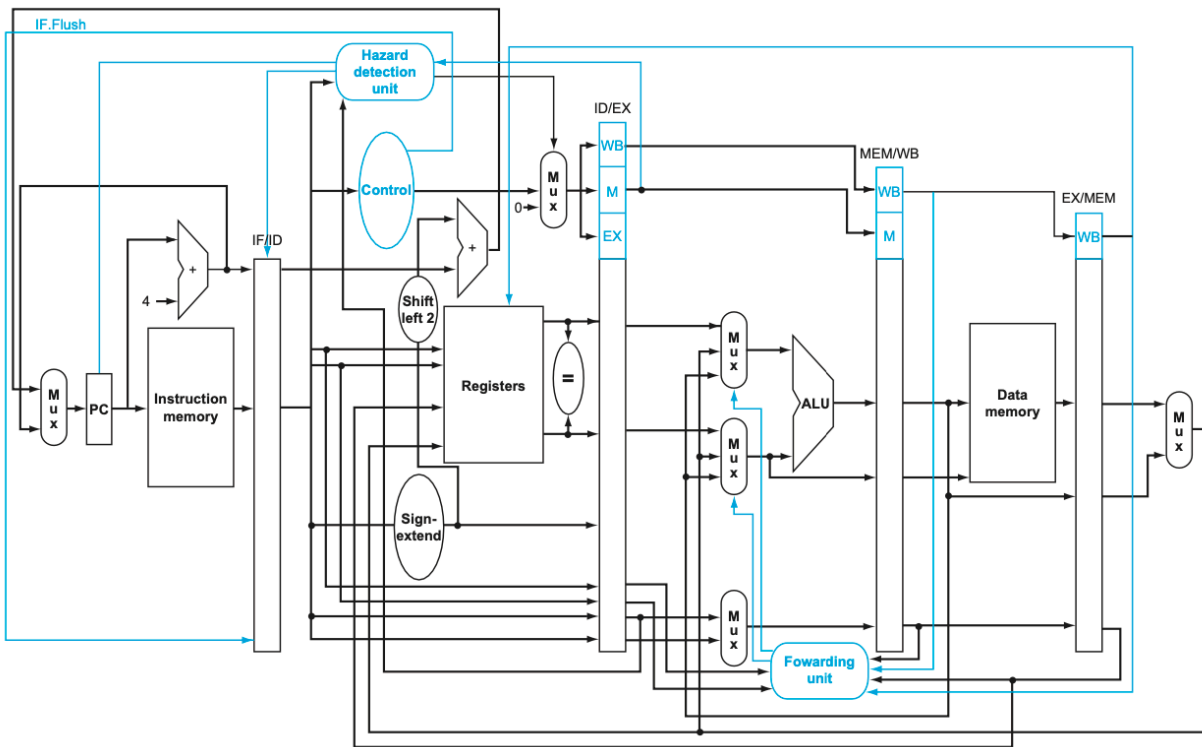
Word 0	Word 1	Word 2	Word 3	Word 4	Word 5	Word 6	Word 7
{ $i==0$ M}		{ $i==0$ S}, { $i==1$ T}		{ $i==1$ S}, { $i==2$ T}		{ $i==2$ S}, { $i==3$ T}	
{ $i==3$ M}, { $i==4$ T}		{ $i==4$ S}, { $i==5$ T}		{ $i==5$ S}, { $i==6$ T}		{ $i==6$ S}	

2M, 6T, 6S

(Intentionally blank, in case you need extra space for work/answers)

Problem 8 (8 marks)

Consider COD Figure 4.65, a 5-stage pipelined processor with **Forward-to-EX**, **RF Bypass**, **Branches Resolved in Decode**, and a **predict-not-taken** branch policy.



You run the following MIPS program on this processor. Be sure to **follow the branch resolutions in L3 and L5**, and **consider register dependences that generate stalls**.

```

L1: lw    $t0, 12($s0)
L2: beq   $t0, $zero, L6    <not taken first time, always taken after>
L3: addi  $t0, $s5, 13
L4: beq   $s4, $s5, L2      <taken every time>
L5: xor   $s5, $t7, $zero
L6: sub   $s1, $t5, $t6
    
```

Complete the **pipeline diagram** below showing the execution (use a * like problem 6 to denote a stall if necessary) until L6 hits the WriteBack stage.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
L1	F	D	X	M	W														
L2		F	D*	D*	D	X	M	W											
L3			F*	F*	F	D	X	M	W										
L4						F	D	X	M	W									
L5							F	=											
L2								F	D	X	M	W							
L3									F	=									
L6										F	D	X	M	W					

L2 uses the result of L1's loadword. L2 must stall 2 cycles because branch inputs can only benefit from RF bypass (W→D). Since processor is predict-not-taken, every branch taken (L4 and L2 the 2nd time) causes the instruction after the branch (L5 and L3 the 2nd time) to be flushed when it hits the Decode stage.

This page provides blank pipeline diagrams, in case you need them.

[illegible]

General Comments

- Pencils are better than Pens if you ever change your mind after writing.
- MIPS ASM
 - branches should use Labels. If you insist on offset integers, it's based on the next instruction (e.g. PC+4)
 - labels, like L1, LOOP, or EXIT, need to end in colon (they're syntax, not comments) e.g. `L1: sll $zero, $zero, 0`
 - Labels are only required when they're the destination target of a branch; optional otherwise. We use them in lectures to be able to easily reference them.
 - Use commas between "arguments" to opcodes. e.g. `sll $zero, $zero, 0`
 - Commands don't end in ; in MIPS, though C and C-like languages use them.
- Hazards
 - If L2 writes \$t0, L4 writes \$t0, and L5 reads \$t0, then there is no L2→L5 RAW hazard as L4 breaks the dependency. L5 will use L4's result, not L2's.
 - `sw $t2, 0($s0)` looks like \$t2 is destination register.
- Computer Scientists and Engineers have often used KB or MB for base-2 numbers for simplicity, when those prefixes are actually base-10. A 4K run is 4000 meters or $4 \cdot 1000^1$. 4 "kilo" bytes is actually $2^{12} = 4096$ bytes, which is more precisely called 4 KiB ($4 \cdot 2^{10}$). On the back of the MIPS Green Card is a section for Size Prefixes, and shows the difference between e.g. Kilo- and Kibi-. Only disk storage vendors intentionally use base-10 numbers because selling you a 1 TB HDD sounds impressive but they'll give you storage for $1 \text{ TB} = 1000^4 = 1,000,000,000,000$ bytes instead of $1 \text{ TiB} = 2^{40} = 1,099,511,627,776$ bytes.